01/04/26

* Multi-class Classification :-

Red - C1
Blue - C2
Green - C3
→ multi class (not binary)

C=3

① One vs Rest (O vs R)

② One vs One (O vs O)

① One vs Rest :-

→ so we converted multiclass ~~Model~~ to binary class.

M1 :- Red vs (others)
or
Red vs (Blue + Green)

so Red will be marked as "1" and rest will be -ve class or "0"

M2 :- Blue vs (others)

M3 :- Green vs (others)

x_1	x_2	y
1	8	Red
2	4.8	Blue
3.1	7.4	Blue
4.8	-4.2	Red
1.2	2.2	Green
2.1	-1.8	Blue

Test data ⇒ 2.1, 8.4 ⇒ C = ? which class?

"n" models for "n" classes.

These are binary classificatⁿ Models.

* The question we ask :- When we get new test data, which class do we assign it to?
∴ predict $\hat{y}$ given the test sample

$$X = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_d \end{bmatrix} \Rightarrow \hat{y} = ?$$

Date

Logistic regression: Probability score → give threshold: then we get labels
Decision trees: Labels.

* Score based models → which ones have we learnt?

→ If our model gives us prob. scores as Red ⇒ 0.7, Blue ⇒ 0.2, Green ⇒ 0.9, we take Green as our label as it gives highest score and therefore highest confidence

→ O vs R relies on some kind of score to give out an output label similar to how important a particular node is in a classifier
Eg in SVM → margin is a score/metric

(highest score) $\arg\max_{S}$ (m score) → This is given as the label

* In classifiers we use some "metric" as a score.

② One vs One (O vs O)

M1: Red vs Blue → here we are removing rows with green and so on for each
∴ our dataset is getting shrunk this does not happen in OVR.

M2: Red vs Green

M3: Blue vs Green

* Handshake problem: If n classes, then how many models can I build? nC_2
∴ Higher models

→ In pre-processing ⇒ we will remove extra rows
→ Then we will train the model → build the model
Now in our test phase, we want to predict $\hat{y}$. Therefore we pass through each model and take majority class (max voting)
↳ can also do → probability scoring
→ sum of prob. ∝ prob. for a label

→ we also need to normalize

✱ Feature selection :

- minimal setup features that are good enough for a model.
  ↳ we need minimal features to :
  (1) reduce computational complexity

• No. of samples << No. of features

S_1 x_1, x_2, x_3 ------ $x_{100,000} \rightarrow y^{(1)}$

S_2 x_1, x_2, x_3 ------ $x_{100,000} \rightarrow y^{(2)}$

S_3 x_1, x_2, x_3 ------ $x_{100,000} \rightarrow y^{(3)}$

✱ If more features, less samples : overfitting / depending
✱ Ideal no. of samples
ML : 10 × No. of features
DL : 10000 × No. of features

✱ Strategies :

→ ~~Do~~ LASSO / L1 Regularisation
→ Forward Search
→ Backward Search
→ Recursive Feature Elimination (RFE)